

CSC270 · DATABASE SYSTEMS · LECTURE 25

Physical Design — Query Performance

# Indexing

CSC270 · Database Systems — COMSATS University Islamabad, Lahore Campus

# Recap: Where We Are

- Units 3–5 covered relational design — normalization, ER/EER modeling, choosing the right tables and columns.
- Every one of those well-designed tables still lives on disk as physical blocks, read one block at a time.
- We have quietly assumed the DBMS finds rows efficiently — this lecture asks what actually happens on disk.
- Without help, the DBMS has exactly one option: check every single row, every time.
- Indexing is the physical-design technique that avoids this — arguably the single most consequential decision for query performance.
- By the end of this lecture: how indexes are structured, and how to create the right ones in real SQL.

# Agenda

- 1 Why no index forces a full table scan
- 2 Primary indexes — built on the file's ordering key
- 3 Secondary indexes — for fields the file isn't sorted by
- 4 Dense vs. sparse indexes
- 5 Single-level vs. multi-level indexes (B-trees)
- 6 Real CREATE INDEX SQL syntax
- 7 Choosing which columns are worth indexing
- 8 The trade-off: faster reads, slower writes

# The Cost of Not Having an Index

PropertyForRent with 10,000,000 rows across 500,000 disk blocks, and a query for propertyNo = 'PG21'.

## WITHOUT AN INDEX — FULL TABLE SCAN

### Read block 1

Check every row — no match

### Read block 2

Check every row — no match

### ... up to block 500,000

Worst case: read every block

## WITH AN INDEX — DIRECT LOOKUP

### Look up PG21 in the index

A handful of comparisons, not 500,000

### Follow the pointer

Jump straight to block 214,009

#### DEFINITION

An index is a small, separate structure mapping field values directly to the block/row that holds them — the DBMS jumps to the answer instead of scanning for it.

## WORKED EXAMPLE

# Full Table Scan — Step by Step

Same PropertyForRent table — 10,000,000 rows across 500,000 disk blocks. Here is exactly what a full scan costs.

### LOOKING UP `propertyNo = 'PL94'` (LUCKY CASE)

#### Blocks 1 through 214,008



Checked in full — no match in any of them



#### Block 214,009



Match found here — the scan can finally stop



#### 214,009 blocks read



Far better than worst case — but not something you can plan around

### LOOKING UP A ROW THAT DOES NOT EXIST

#### Blocks 1 through 500,000



Every block must be checked — there is no early exit



#### 500,000 blocks read



The true worst case for a full table scan



#### ON AVERAGE

Averaged over many different queries, a full table scan reads about half the table — roughly 250,000 blocks here — because a match is equally likely to live in any block. This  $O(n)$  cost is exactly what an index exists to avoid.

# Primary Indexes

Built on the field the data file is physically ordered by — almost always the primary key. One entry per distinct key value, pointing to the first record with that value.

Primary index on PropertyForRent(propertyNo) — file ordered by propertyNo

| Index entry: propertyNo | Pointer   |
|-------------------------|-----------|
| PA14                    | → block 1 |
| PG4                     | → block 2 |
| PG21                    | → block 2 |
| PG36                    | → block 3 |
| PL94                    | → block 3 |

**ASSUMES AN ORDERED FILE**

This only works because the file is physically stored in propertyNo order. If unordered (the common case), no primary index is possible — you'd need a clustering index or a secondary index instead.

# Secondary Indexes

Most useful queries filter on a field the file isn't ordered by — e.g. “find all properties in Lahore.” A secondary index must have one entry for every record, since matching rows can be scattered anywhere in the file.

Secondary index on PropertyForRent(city)

| Index entry: city | Pointer |
|-------------------|---------|
| Islamabad         | → PA14  |
| Karachi           | → PL94  |
| Lahore            | → PG4   |
| Lahore            | → PG36  |
| Lahore            | → PG21  |

**WHY LAHORE REPEATS**

Lahore appears three times — once per matching record — because a secondary index can't compress multiple rows into one entry the way a primary index can. This is the direct cause of the dense-vs-sparse distinction next.

# Primary vs. Secondary Indexes

Two different answers to “which field is this index built on?” — side by side before we look at index shape next.

Primary index vs. secondary index, compared

| Primary Index                              | Secondary Index                             |
|--------------------------------------------|---------------------------------------------|
| Built on the file's ordering field         | Built on any other, non-ordering field      |
| One entry per distinct key value           | One entry per record — always dense         |
| Can be sparse — one entry per block        | Always dense — one entry per record         |
| At most one per table (the ordering field) | Many per table — one per useful query field |
| Example: index on propertyNo               | Example: index on city                      |

**WHY THIS MATTERS**

A table always has at most one primary (sparse) index — on its physical ordering field — but can have many secondary (dense) indexes, one for each field your queries actually filter on.

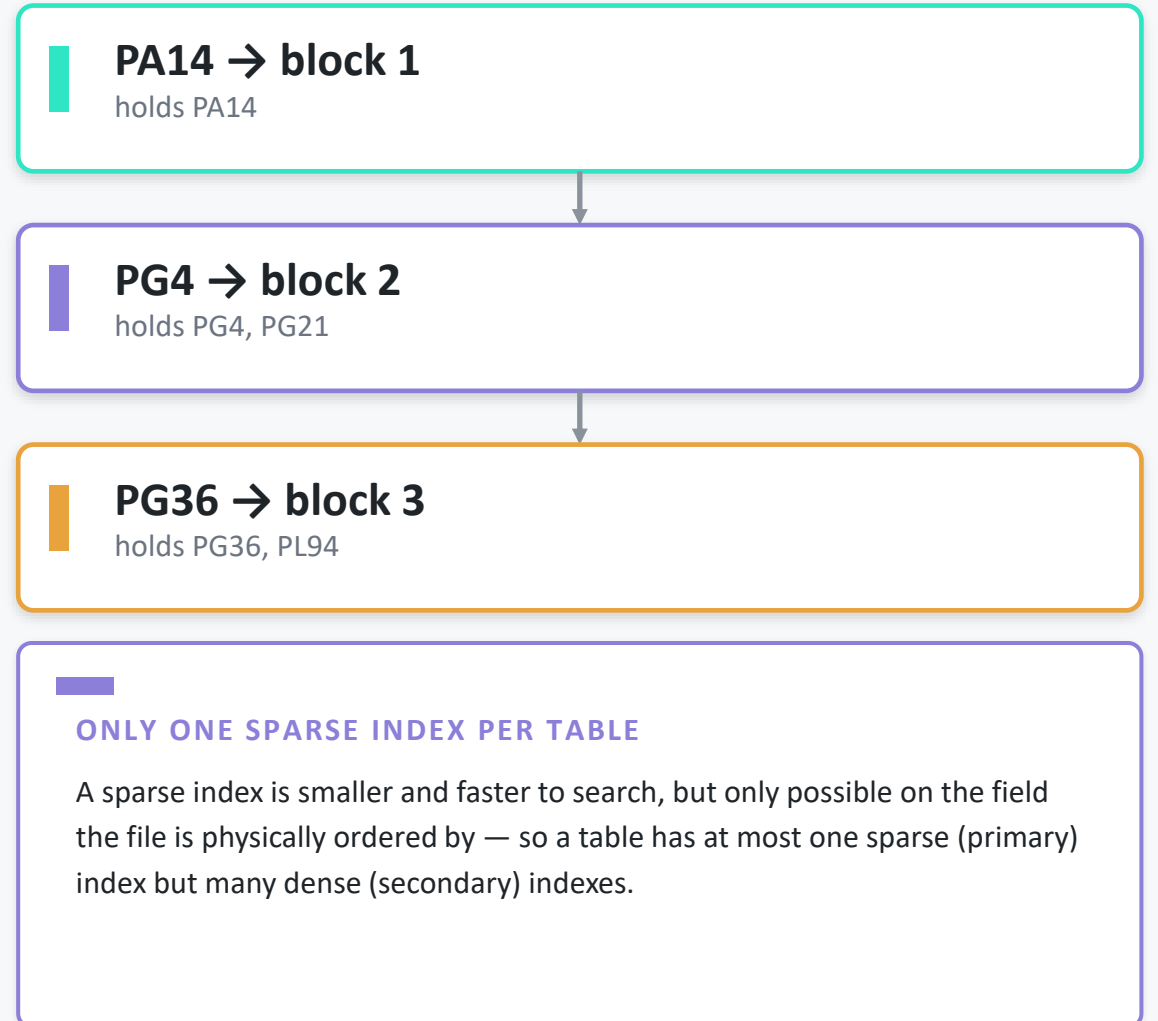


# Dense and Sparse Indexes

## DENSE INDEX — 1 ENTRY / RECORD



## SPARSE INDEX — 1 ENTRY / BLOCK



## Single-Level Indexes — The Scaling Problem

Every index so far is one flat, sorted list. That works only while the index stays small.

### A small index: a few thousand rows

Fits comfortably in memory — only a handful of blocks to search

### A large index: 10,000,000 rows

10,000,000 index entries — now a multi-megabyte file that no longer fits in memory

### Searching the index becomes the bottleneck

Exactly the problem indexing was supposed to solve — now one level up, inside the index itself

### THE INDEX NEEDS ITS OWN INDEX

If a single-level index is itself too big to search quickly, the fix is to apply the same idea recursively: build an index on the index. That is exactly where we go next.

# Multi-Level Indexes and B-Trees

An index over ten million rows is itself a large file. The fix: build an index on the index — a multi-level index, the structure underlying every production B-tree/B+-tree.

## Top-level (second-level) index

Small enough to fit in one block — one entry per block of the level below

## First-level index

One entry per data block, exactly like a single-level index

## Data file

The actual PropertyForRent rows, in disk blocks

### WHY THIS IS $O(\log N)$

Each level divides the search space by roughly the same factor — the entries that fit in one block. That's exactly what gives B-tree indexes their logarithmic lookup cost, versus a full scan's linear  $O(n)$  cost, even as a table grows into the billions of rows.

# B-Tree Lookup, Step by Step

A 2-level B-tree over 8 property keys, root separators [PG21, PX02]. Trace a lookup for propertyNo = 'PL94'.

LOOKING UP propertyNo = 'PL94'

## Step 1 — Compare at the root

Root separators [PG21, PX02]:  $PL94 \geq PG21$  and  $< PX02$

## Step 2 — Descend to the middle child

Follow the middle pointer — the 2nd block read

## Step 3 — Scan the leaf block

Leaf holds PG21, PG36, PL94 — found, with a row pointer

TOTAL COST: 2 BLOCK READS

## Root block

1 read — compared against 2 separator keys

## Leaf block

1 read — compared against up to 3 keys

### WHY THIS SCALES

Each extra level the tree grows adds exactly one more block read, but multiplies the number of keys it can index by the branching factor — so a B-tree over billions of rows still resolves a lookup in a handful of reads.

## B-Tree vs. B+-Tree

### ● B-Tree: data in every node

Keys and row pointers live in both internal nodes and leaves

### ● B+-Tree: data only in leaves

Internal nodes hold only separator keys, used purely for navigation

### ● B+-Tree leaves are chained

Leaf blocks are linked in a list, so a range scan never revisits the root

### ● Nearly every production DBMS uses B+-trees

MySQL InnoDB, PostgreSQL, SQL Server, and Oracle all index this way

#### WHY RANGE QUERIES LOVE B+-TREES

A query like `rent BETWEEN 20000 AND 30000` finds the first matching leaf, then walks the leaf chain forward — no need to revisit the tree at all. That is why B+-trees, not plain B-trees, dominate real-world index implementations.

## Other Real-World Index Types

### ● Hash Index

$O(1)$  exact-match lookups via a hash function — useless for range queries or sorting

### ● Bitmap Index

One bit per possible value per row — excels on low-cardinality columns like `isActive`

### ● Full-Text Index

Tokenizes text into searchable words — powers `LIKE %word%` and real search features

### ● Spatial (R-Tree) Index

Indexes 2D/3D regions — powers “find all branches within 5km” queries

#### THE B-TREE IS STILL THE DEFAULT

Unless your workload has a specific reason to reach for one of these — pure equality lookups, a handful of distinct values, free text search, or geographic data — a B+-tree remains the right first choice for almost every column.

## Creating Indexes in SQL

You never build index structures yourself — the DBMS does it once you name the column(s).



SQL

```
-- A simple secondary index on one column
CREATE INDEX idx_property_city
ON PropertyForRent (city);

-- A unique index -- also enforces that no two rows may share a value
CREATE UNIQUE INDEX idx_property_no
ON PropertyForRent (propertyNo);

-- A composite (multi-column) index -- useful for queries filtering on both
CREATE INDEX idx_property_city_rent
ON PropertyForRent (city, rent);

-- Removing an index that is no longer earning its cost
DROP INDEX idx_property_city;
```

## Query Plan: With vs. Without an Index

Same query, same table — before and after CREATE INDEX idx\_property\_city, read with EXPLAIN.



EXPLAIN

```
-- BEFORE: no index on city
EXPLAIN SELECT * FROM PropertyForRent WHERE city = 'Lahore';
    Seq Scan on PropertyForRent  (cost=0.00..180000.00 rows=2000000)

-- AFTER: CREATE INDEX idx_property_city ON PropertyForRent(city);
EXPLAIN SELECT * FROM PropertyForRent WHERE city = 'Lahore';
    Index Scan using idx_property_city  (cost=0.43..420.18 rows=2000000)

-- The planner switches strategy automatically once the index exists
Seq Scan cost ≈ 180,000    vs.    Index Scan cost ≈ 420
→ roughly a 400x drop in estimated cost for this filter

-- The table was not touched — only the access path changed
No application code changes; the DBMS just picks the faster plan
```



# Index Selection and Performance

## ● High selectivity

Many distinct values (propertyNo, email). A 2-value column like isActive barely benefits.

## ● Foreign keys

Almost always worth indexing — constantly used in joins.

## ● Columns in WHERE clauses

Filters run often, against large tables, are the clearest case.

## ● Columns in ORDER BY / JOIN

An index on the sort/join column can skip a separate sort step entirely.

### INDEXING EVERYTHING IS NOT A SHORTCUT

Every index must be updated on every INSERT/UPDATE/DELETE that touches the column — over-indexing slows writes dramatically without benefit. Index selection is a real design decision driven by the actual query workload, not “more is always better.”

# Composite Indexes and Column Order

A composite index on (city, rent) is not the same as one on (rent, city) — column order decides which queries it can actually serve.

Index on PropertyForRent(city, rent) — the “leftmost prefix” rule

| Query filters on...              | Can this index help?                    |
|----------------------------------|-----------------------------------------|
| city = 'Lahore'                  | Yes — city is the leftmost column       |
| city = 'Lahore' AND rent > 30000 | Yes — uses both columns, in order       |
| rent > 30000 (no city filter)    | No — skips the leftmost column entirely |
| city = 'Lahore' ORDER BY rent    | Yes — the sort order matches too        |
| rent > 30000 AND city = 'Lahore' | Yes — the DBMS reorders filters for you |

THE LEFTMOST-PREFIX RULE

A composite index on (city, rent) serves any query filtering on city alone, or on city then rent together — but never a query that filters on rent alone, since the index is physically sorted by city first. Put your most-filtered column first.

# Advantages and Disadvantages of Indexing

## ADVANTAGES

### Much faster reads

  $O(\log n)$  lookups instead of  $O(n)$  scans

### Faster sorting and joins

 An indexed sort order can avoid a separate sort step

### Enforces uniqueness

 UNIQUE indexes double as a constraint

## DISADVANTAGES

### Slower writes

 Every INSERT/UPDATE/DELETE must also update every affected index

### Extra storage

 An index over a large table can itself be sizeable

### Maintenance overhead

 Indexes can fragment and need periodic rebuilding

# Common Indexing Mistakes

## DO

### Index foreign keys and WHERE/JOIN columns

These are used constantly — the clearest win for the storage cost

### Measure with EXPLAIN before and after

Confirm the planner actually switches to an index scan

### Watch column order in composite indexes

Put the most-filtered, most-selective column first

## DON'T

### Index every column “just in case”

Unused indexes still slow every write with zero read benefit

### Index low-cardinality columns alone

A boolean or 2–3 value column barely narrows a scan

### Forget to monitor index usage over time

Query patterns change — yesterday's useful index can become dead weight

# Key Takeaways

- With no index, the DBMS runs a full table scan for any lookup — cost grows linearly with table size.
- A primary index (one entry per key) needs an ordered file; a secondary index must be dense (one entry per record).
- Sparse indexes (one entry per block) only work on an ordered file's primary index — every secondary index is dense.
- A single-level index that grows too large is solved with a multi-level index — the idea behind B-tree/B+-tree structures.
- Choose indexes from real query workload — selectivity, foreign keys, WHERE/JOIN/ORDER BY columns — never every column.
- Indexing trades faster reads and joins for slower writes and extra storage — never “free” performance.

# Looking Ahead

- Indexing closes out the physical-design toolbox this course builds on top of the relational model.
- The next three lectures step outside the relational model entirely, into NoSQL.
- Lecture 26 asks why relational databases start to struggle at web scale, and introduces the CAP theorem.
- Lecture 27 goes hands-on with MongoDB and the document model — the most widely used NoSQL database today.
- Lecture 28 covers how MongoDB actually scales: replication and sharding.
- The same trade-off you just saw with indexes — spend something to gain speed — resurfaces again and again from here on.